

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2022/2023

SEPTEMBER EXAMINATION

UCCC2063 ALGORITHMS ANALYSIS

WEDNESDAY, 21 SEPTEMBER 2022

TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)
BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)
COMPUTER ENGINEERING

Instruction to Candidates:

This question paper consists of THREE (3) questions in Section A and TWO (2) questions in Section B.

Answer **ALL** questions in **Section A** and **ONLY ONE (1)** question in **Section B**.
Each question carries 25 marks.

Should a candidate answer more than ONE (1) question in section B, marks will only be awarded for the FIRST ONE (1) question in that section in the order the candidate submits the answers.

Candidates are NOT allowed to use calculator.

Answer questions only in the answer booklet provided.

UCCC2063 ALGORITHMS ANALYSIS**SECTION A (Answer All Questions)**

Q1. [Asymptotic Notations]

(a) Express the following functions in terms of Ω notation.

(i) $n^{1.8} \log n^{1024} + n^2$ (1 mark)

(ii) $(700n)^{9988} + 460n(\log_2 n)^{1238} + 0.1(2^n) + 2^{1.001n}$ (1 mark)

(iii) $15(n+1)(n-1)! + 2^{256n} + n^{40} + n!$ (1 mark)

(iv) $4n^2 + 9n^{1.9} \log_2 n + 2047$ (1 mark)

(v) $\log_{10} n^{2048}$ (1 mark)

(vi) $\log_{100} n^{1024} + \log^{32} n$ (1 mark)

(b) Using the definition of Big-Oh, prove $2n = O(n^{1.2})$. (6 marks)
[Hint: $O(f(n)) = \{g(n) \mid \exists c, n_0 \bullet \forall n > n_0 \bullet g(n) \leq c f(n)\}$](c) Using the definition of Ω , prove $2n \neq \Omega(n^{1.2})$. (6 marks)
[Hint: $\Omega(f(n)) = \{g(n) \mid \exists c, n_0 \bullet \forall n > n_0 \bullet g(n) \geq c f(n)\}$](d) Using the definition of Big-Oh, prove $O(n) = O(2n)$. (7 marks)
[Hint: Prove $O(n) \subset O(2n)$, and $O(2n) \subset O(n)$.]

[Total : 25 marks]

UCCC2063 ALGORITHMS ANALYSIS

Q2. [Solving Recurrence]

(a) Use master method to solve the following recurrences.

(i) $T(n) = 5T(n/5) + n \log n.$ (4 marks)

(ii) $T(n) = 5T(n/5) + n^{0.9} \log n$ (4 marks)

(iii) $T(n) = 5T(n/5) + n^2 \log n.$ (4 marks)

(b) Consider the recurrence $T(n) = 5T(n/5) + n.$ (i) Solve $T(n)$ by recursive method (also called expansion). Give your answer in Θ notation. (4 marks)(ii) Solve $T(n)$ by substitution (also called Mathematical Induction). Give your answer in Big-Oh notation. (4 marks)

[Hint: The result must be consistent with (b)(i).]

(c) Give a recurrence which can be solved by substitution (Mathematical Induction), but not master method and recursive method (expansion). Solve your recurrence by substitution (Mathematical Induction), and explain why it is not feasible to be solved by master method and recursive method (expansion). (5 marks)

[Total : 25 marks]

UCCC2063 ALGORITHMS ANALYSIS

Q3. [Binary Tree]

The **struct** of a node of a binary tree is stated below.

```
struct node{
    int element;
    node* left;
    node* right;
    node* parent;
    int distance;
};
```

Variable **element** stores the data, **left** and **right** are pointers pointing to the left and right subtrees, respectively, and **parent** is a pointer to the parent, if any. Variable **distance** is recursively defined as follows.

The **distance** of the root is defined to be 0. For any other node, its **distance** is one plus the **distance** of its parent. Note that **distance** is undefined for empty trees.

- (a) Using the above definition, write a recursive algorithm in pseudocode format for filling up the **distance** of each node in a binary tree. State clearly the pre-condition(s) of the algorithm. (8 marks)
- (b) The leftview of a binary tree is $a_0, a_1, \dots, a_M$, where M is the maximum **distance**, and for $i \in [0, M]$, a_i is the **element** of the leftmost node with **distance** = i . E.g. the binary tree in Figure 3.1 below shows the **elements** of each node, and the tree has a leftview 4, 6, 8, 3, 1.

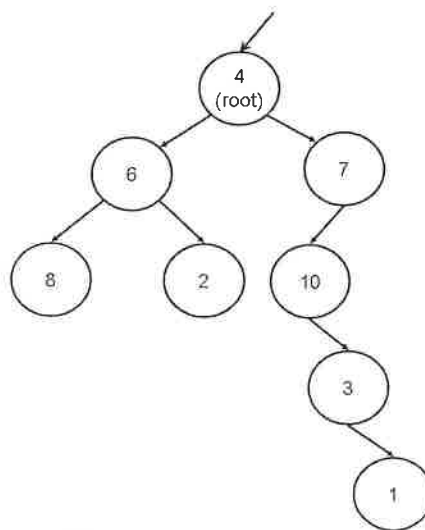


Figure 3.1: A binary tree.

- (i) Give a binary SEARCH tree with its leftview increasingly ordered. (2 marks)

UCCC2063 ALGORITHMS ANALYSIS**Q3. (b)(Continued)**

- (ii) Give a binary SEARCH tree with its leftview decreasingly ordered.
(2 marks)
 - (iii) Give an algorithm to accept a binary tree (practically the root pointer in the parameter) and output its leftview.
(8 marks)
[Hint: Modify the algorithm in (a).]
 - (iv) If the input is always a full binary tree, give a more time-efficient algorithm to output its leftview, and explain the time complexity.
(5 marks)
- [Total : 25 marks]

UCCC2063 ALGORITHMS ANALYSIS**SECTION B (Choose Any One Question)**

Q4. [Design and Analysis of Algorithm]

- (a) A software engineer has an $O(n \log n)$ -time function $\text{SORT}()$ at hand for sorting an array of n integers. The function is correct when n is a power of 2, and it may give a wrong output when n is not a power of 2.

Suggest him a way for using $\text{SORT}()$ to sort n integers within $O(n \log n)$ time, where n is an arbitrary positive integer, and prove the time complexity.

(12 marks)

- (b) You are given a set of positive integers $\{w_1, w_2, \dots, w_p\}$, a positive integer N , and the recursive function $\text{cost}()$, below.

```
For  $i=0$  or  $n=0$ ,  $\text{cost}(i,n)=0$ ;
for  $w_i > n$ ,  $\text{cost}(i,n)=\text{cost}(i-1,n)$ ;
else,  $\text{cost}(i,n)=\max(w_i+\text{cost}(i-1,n-w_i), \text{cost}(i-1,n))$ ;
```

where $i \leq p$, and $n \leq N$.

- (i) Give a recursive algorithm for computing $\text{cost}(p,N)$. (5 marks)
- (ii) Give an $O(pN)$ -time algorithm for computing $\text{cost}(p,N)$. (8 marks)

[Total : 25 marks]

UCCC2063 ALGORITHMS ANALYSIS

Q5. [Graph Algorithms]

Consider an undirected graph $G(V,E)$ with n vertices and e edges, and answer all the questions below.

- (a) Describe adjacency list, and state how it is used for storing $G(V,E)$. (4 marks)
- (b) In terms of n and e , what is the size of the adjacency list for storing the graph G ? Explain. (4 marks)
- (c) Describe a text file format for storing the adjacency list of the graph G , and give a time-efficient algorithm for retrieving the adjacency list from the file. (6 marks)
- (d) The Breadth First Search (BFS) algorithm is given below.

```

BFS( $G, s$ )
for each  $u \in V - \{s\}$  do
    color[ $u$ ]  $\leftarrow$  WHITE
color[ $s$ ]  $\leftarrow$  GRAY
 $Q \leftarrow \{s\}$ 
while  $Q \neq \emptyset$  do (*)
     $u \leftarrow \text{head}[Q]$ 
    for each  $v$  in Adj[ $u$ ] do **
        if color[ $v$ ] = WHITE then
            color[ $v$ ]  $\leftarrow$  GRAY
            ENQUEUE( $Q, v$ )
    DEQUEUE( $Q$ )
    color[ $u$ ]  $\leftarrow$  BLACK

```

The input is (G, s) where G is the graph stored in adjacency list Adj[], and s is the starting vertex. Q is a queue, and head[Q] is the first element in Q . ENQUEUE and DEQUEUE are operations for appending an element to the tail and taking away an element from the front, of the queue, respectively.

- (i) In terms of n and/or e , what is the maximum number of iterations for executing the outer loop (labeled with (*))? Explain. (4 marks)
[Hint: consider the content of the queue.]
- (ii) In terms of n and/or e , summing up for ALL iterations of the outer loop, what is the total number of iterations for executing the inner loop (labeled with (**))? Explain. (4 marks)

UCCC2063 ALGORITHMS ANALYSIS**Q5. (d)(Continued)**

- (iii) In terms of n and/or e , what is the time complexity of the Breadth First Search algorithm? Explain. (3 marks)

[Hint: it is wrong to use the product of the answers of (d)(i) and (d)(ii).]

[Total : 25 marks]
